Lecture – 9 Complexity and Analysis of Algorithms - Part A

M.G.Noel A.S Fernando (PhD)

Professor

Dept. of Information Systems Engineering, UCSC

What is an Algorithm?

- An **algorithm** is a **finite, step-by-step sequence of well-defined instructions** that takes some input, processes it, and produces an output to solve a specific problem.
- Key Characteristics of an Algorithm:
  - **Finiteness** – It must complete after a finite number of steps.
  - **Definiteness** – Each step must be clear, precise, and unambiguous.
  - **Input** – Zero or more inputs are given before or during execution.
  - **Output** – Produces at least one output (the solution to the problem).
  - **Effectiveness** – Each step should be simple enough to be carried out within finite time using basic operations

What is the Running Time of a Program?

What is the Running Time of a Program?

- **Definition:** The *running time* (or *execution time*) of a program is the total time it takes for a computer to execute that program from start to finish for a given input.
- Depends on Input Size (n):
  - Running time usually increases as the size of the input grows.
  - Example: Searching in a list of 10 items is faster than in a list of 1,000 items
- Measured in Steps, Not Seconds:
  - instead of actual clock time (which depends on CPU, compiler, OS), we count the **number of basic operations** (comparisons, assignments, arithmetic, etc.).
  - This makes the analysis machine-independent.
- Types of Running Time Analysis:
  - **Worst Case:** Maximum time taken for any input of size *n*.
  - **Best Case:** Minimum time taken.
  - **Average Case:** Expected time over all possible inputs.
- Relation to Time Complexity:
  - Running time is expressed asymptotically (using Big-O, Big-Ω, Big-Θ) to describe how it grows as input size increases.

What is the Running Time of a Program? Cont..

- How is Running Time Measured?
- There are two perspectives:
  - **Actual running time** — measured in seconds, milliseconds, or nanoseconds when you run the program on real hardware.
  - **Theoretical running time** — expressed using *Big O* notation (e.g., O(n), O(n^2), O(log n)), which describes how the time grows with input size.
- **Theoretical running time** is another way of saying **time complexity**.

Different types of complexities

Big O	Description	Example
O(1)	Constant time	Access an array element
O(log n)	Logarithmic time	Binary search
O(n)	Linear time	Linear search
O(n log n)	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort, nested loops
$O(2^n)$	Exponential	Solving Traveling Salesman by brute force
O(n!)	Factorial	Generating all permutations

What are the factors depending on the running time of a program?

- **The running time (or execution time) of a program** depends on several key factors:
- **algorithmic,**
- **hardware, and**
- **environmental factors**

Algorithmic Factors

- These are related to how you design and write the program.
- **Algorithm efficiency:** The choice of algorithm has the biggest impact.
  - Algorithms with quadratic time complexity $O(n^2)$(like Bubble Sort, Selection Sort, or Insertion Sort) are indeed slower than algorithms with O(n log n) time (like Merge Sort) for large inputs.
  - Input size (n)
- **Implementation details:**
  - The way code is written affects execution speed, for example by using efficient data structures (such as hash tables instead of arrays) and minimizing redundant calculations
  - **Programming language and compiler optimizations:**
  - Middle level languages like C can be faster than Python (high level Language) because they are compiled and closer to machine instructions.

Hardware Factors

- **Processor (CPU) speed and number of cores:**
  - Faster CPUs and parallel cores can execute instructions more quickly or in parallel.
- **Memory (RAM) size and speed:**
  - Insufficient RAM can cause a program to use slower disk storage (paging/swapping).
- **Cache size:**
  - CPUs have fast memory caches; programs that make good use of them run faster.
- **Storage speed:**
  - For programs that read/write lots of data, SSDs are faster than HDDs.

Environmental / System Factors

- **Other running processes:**
  - Other programs can compete for CPU, RAM, or disk I/O.
- **Operating system overhead:**
  - Context switching and multitasking introduce overhead.
- **Network conditions:**
  - For programs that depend on internet or network access, bandwidth and latency matter.
- **Power management:**
  - Laptops or mobile devices might reduce CPU speed to save battery.

What is *Complexity analysis* ?

- **Complexity analysis** is defined as a technique to characterize the time taken by an **algorithm** with respect to input size (independent from the machine, language and compiler).
- It is used for evaluating the **variations of execution time** on different algorithms.

What is the need for Complexity Analysis?

- Complexity Analysis determines the amount of time and space resources required to execute it.
- It is used for comparing different algorithms on different input sizes.
- Complexity helps to determine the difficulty of a problem.
- Often measured by how much time and space (memory) it takes to solve a particular problem

Problem solving

- To solve a problem, more than one algorithm can be used.
- How should we choose the best algorithm to solve a given problem?
- As a general rule, we should always pick an algorithm that is easy to understand, implement, and document.
  - When performance is essential, we must choose an algorithm that runs quickly and uses the available computing resources efficiently.

Choosing an Algorithm

- When you need to write a program(algorithm) that is to be used and maintained by many people over a long period of time, other issues arise.
- **Simplicity** : One is the understandability, or Simplicity, of the underlying algorithm.
  - E.g. a simple algorithm is easier to implement correctly than a complex one.
- **Clarity** :Programs should be written **clearly** and documented carefully so that they can be maintained by others.
  - With good documentation, modifications to the original program can conveniently be done by someone other than the original writer.

Choosing an Algorithm

- **Efficiency**: When a program is to be run repeatedly, its efficiency and that of its underlying algorithm become important.
- Generally, we associate efficiency with the time a program takes to run, although there are other resources that a program sometimes must conserve, such as
  - The amount of storage space taken by its variables.
  - The amount of traffic it generates on a network of computers.
  - The amount of data that must be moved to and from disks.

Difference between running time and time complexity

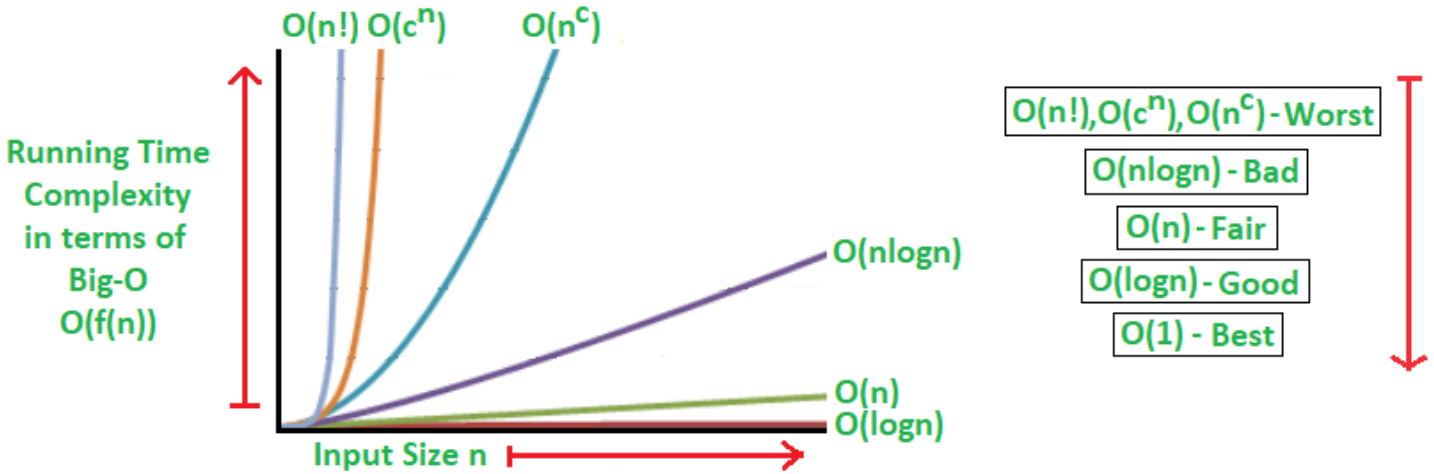
- Running time is how long it takes a program to run.
- Time complexity is a description of the asymptotic behavior of running time as input size tends to infinity.
- **Or in other words**
- Time complexity is a complete theoretical concept related to algorithms, while running time is the time a code would take to run, not at all theoretical

Asymptotic Notations in Complexity Analysis:

- In algorithm analysis, the **upper bound** is represented using **Big-O notation (O)**
- Therefore, it gives the **worst-case** complexity of an algorithm.
- By using **big O- notation**, we can asymptotically limit the **expansion** of a running time to a range of constant factors above and below.
- It is a model for quantifying **algorithm performance.**

Upper Bound of Running Time

The **upper bound** of a program's running time describes the **maximum amount of time an algorithm can take** to complete for an input of size *n*.



ASYMPTOTIC NOTATION

- The performance evaluation of an algorithm is obtained by totaling the number of occurrences of each operation when running the algorithm.
- The performance is evaluated as a function of the input size n.
- Complexity refers to the rate at which the storage or time grows as a function of the problem size. – order of growth of running time of an algorithm
- Asymptotic analysis is based on the idea that as the problem size grows, the complexity can be described as a simple proportionality to some known function.
- Notation used,
  - Big O notation (O)
  - Big Omega (Ω)
  - Big Theta (θ)

Asymptotic Notation

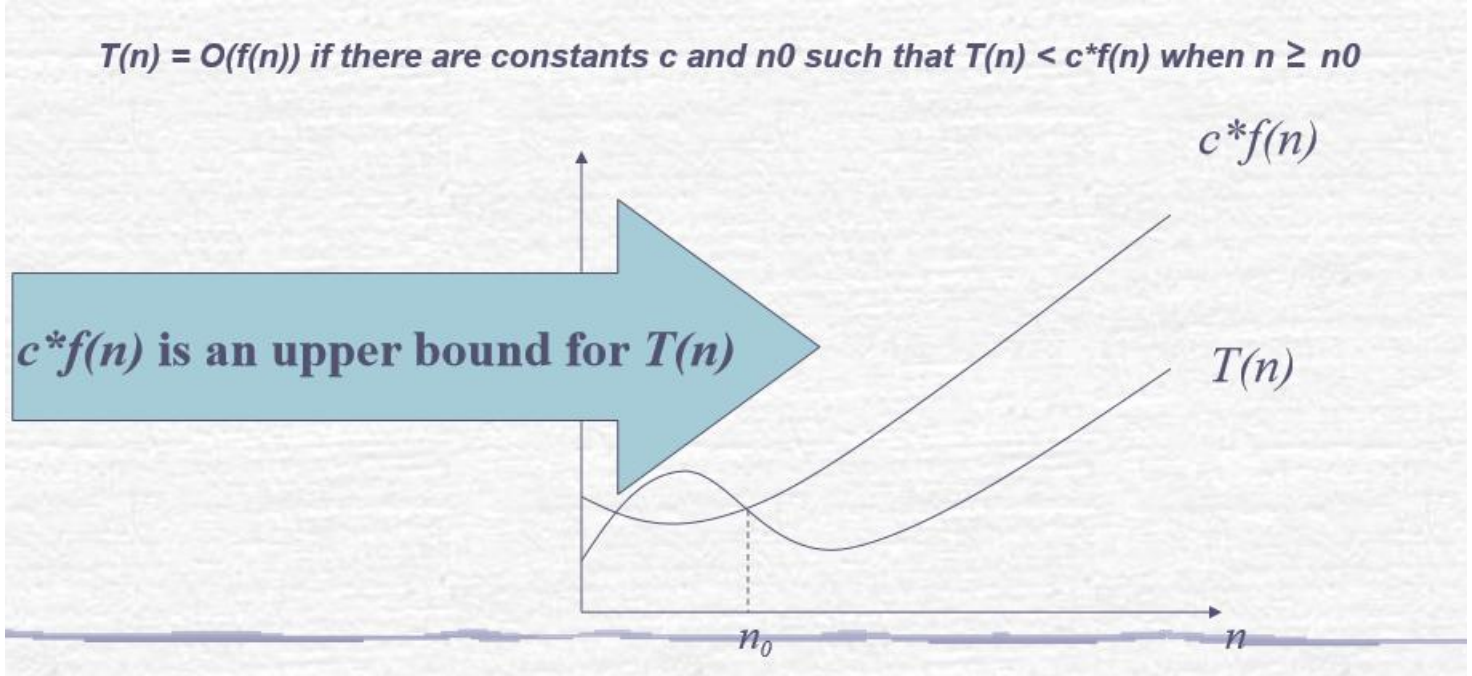
- Describes the behavior of the time or space complexity for large instance characteristic.
- Main Types of Asymptotic Notation
- **Big-O Notation (O) – Upper Bound**
  - Describes the **maximum time** an algorithm can take.
  - Example: Linear Search → **O(n),**Merge Sort → **O(n log n)**
- **Big-Ω Notation (Ω) – Lower Bound**
  - Describes the **minimum time** an algorithm will take (best case).
  - Example: Linear Search → **Ω(1)** (if the element is first),Merge Sort → **Ω(n log n)**
- **Big-Θ Notation (Θ) – Tight Bound (average)**
  - Describes the **exact asymptotic growth** (both upper and lower bounds).
  - Example:Merge Sort → **Θ(n log n),**Bubble Sort → **Θ(n^2)**

Theoretical Analysis of Time Efficiency

- The **theoretical analysis of time efficiency** is the process of evaluating how the running time of an algorithm grows with the size of the input, *without actually implementing or executing it*.
- It is **machine-independent** and focuses on counting **basic operations** (comparisons, assignments, arithmetic) rather than real clock time
- **Worst-case** – The maximum number of steps taken on any instance of size **a (or** T(n) = maximum time of algorithm on any input of size n)
- **Best-case** – The minimum number of steps taken on any instance of size **a**.
- **Average case** – An average number of steps taken on any instance of size **a** (T(n) = expected time of algorithm over all inputs of size n)

Big O - Not a Speed Comparison

- Big-O notation does not directly compare the *speed* of two algorithms.
  Instead, it describes how the number of operations grows as the input size increases, identifying the dominant factor that affects scalability. will grow by as the number of inputs increases.



T(n) and **F(n)** because both are sometimes used in algorithm analysis, but they have slightly different contexts.
T(n): Represents the **total running time** of an algorithm as a function of input size *n*
f(n):Represents the **number of times a specific operation (or a basic operation) is executed**.

Big Oh (O) notation

- Mathematical Representation of Big-O Notation:
- $F(n) \in O(g(n))$ if there exist positive constants c and n0 such that:
- $f(n) \leq c \cdot g(n)$ for all $n \geq n0$
    - **T(n) or f(n)** : Actual number of steps (running time) for input size n.
    - **g(n)**: A simpler function representing the growth rate (like n, n log n, n^2).
    - c: Constant factor scaling g(n) to cover T(n).
    - n_0: Threshold beyond which the inequality holds

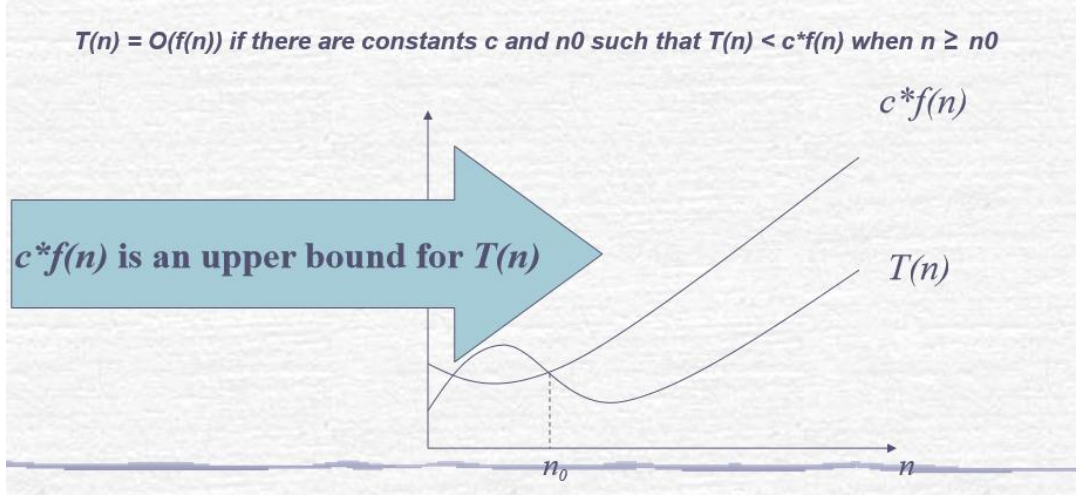
Big-O Notation Definition and Explanation of Each Part

We say

$$f(n) \in O(g(n))$$

if there exist positive constants **c** and **n_0** such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$



1. f(n) or T(n):
   - This is the actual running time (or number of steps) of the algorithm for input size n.
   - Example: $f(n) = 3n^2 + 20n + 5$.

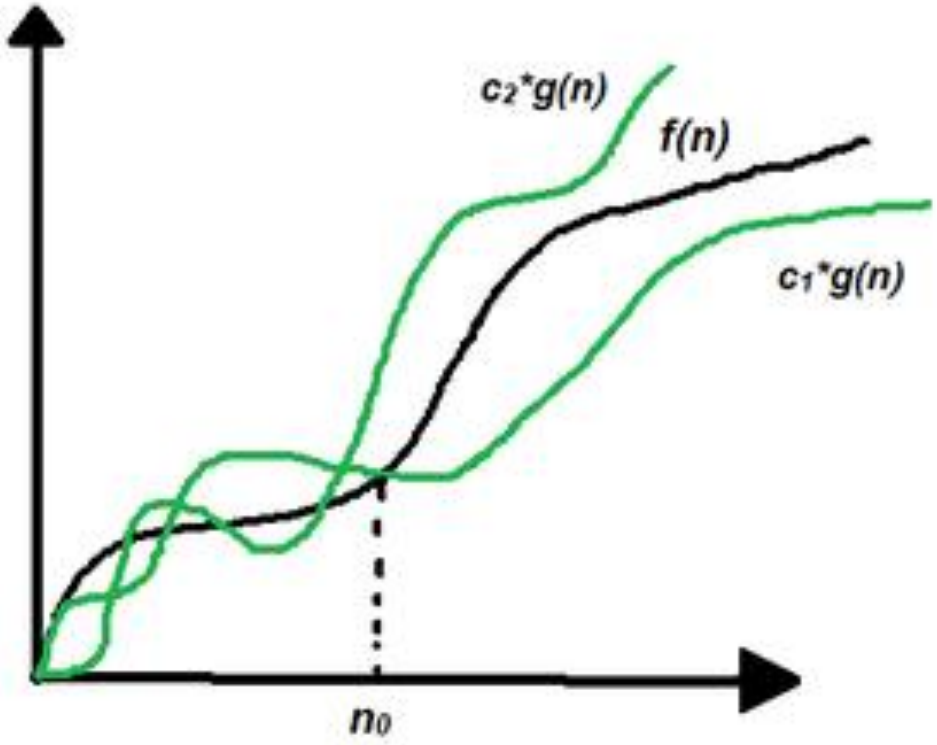
•2. g(n):
   - •A simpler function that describes the growth rate of f(n).
   - •Example candidates: n,nlogn,n^2,n^3
   - •We use it as a benchmark to compare algorithm efficiency.

3. c (constant factor)
   - •A positive scaling constant that ensures $f(n)$never grows faster than $c \cdot g(n)$)after some point).
   - •Example: If $f(n) = 3n^2 + 20n + 5$,then choosing $c = 4$ and $g(n) = n^2$makes the inequality true for large n.

•4. n_0 (threshold):
   - •A cutoff point beyond which the inequality holds.
   - •For smaller n, the inequality may not hold, but Big-O only cares about asymptotic behavior (large n).

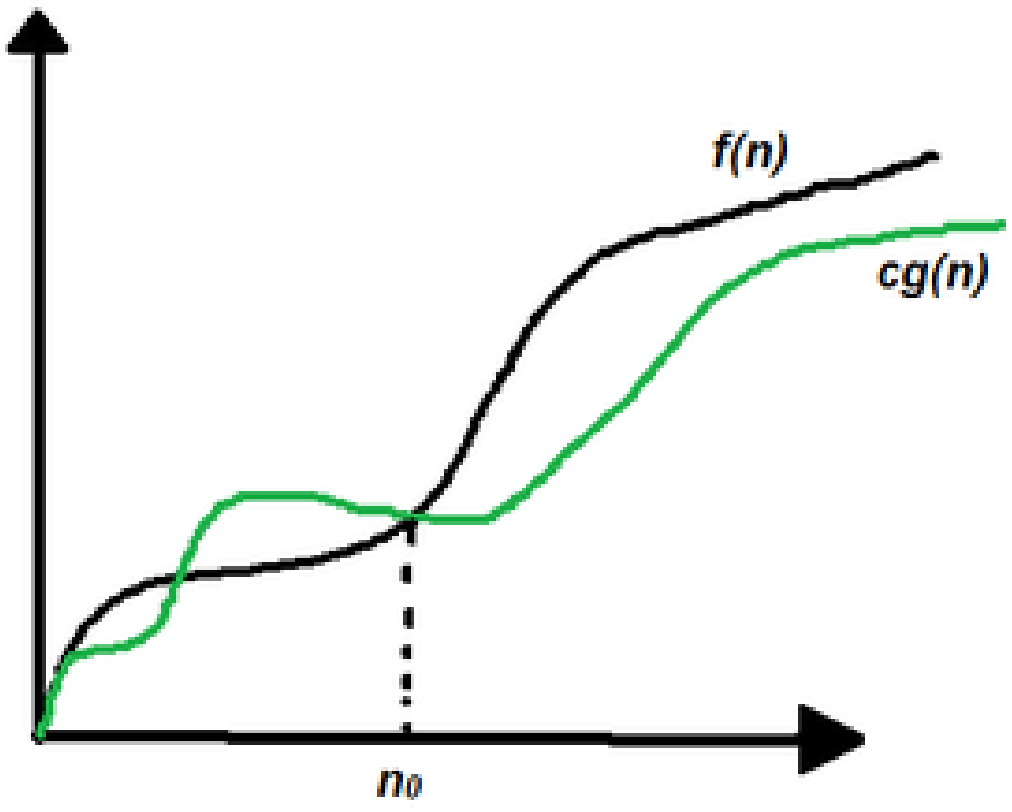
Theta Notation

- it is used
  for **analyzing** the **average-case** complexity of an algorithm.
- Mathematical Representation:
- ***g(n))*** = ***{f(n):*** *there exist positive constants* c_1, c_2 *and* n_0 *such that* $\boldsymbol{0 \le c_1 * g(n) \le f(n) \le c_2 * g(n)}$ *for all* $n \ge n_0$*}*



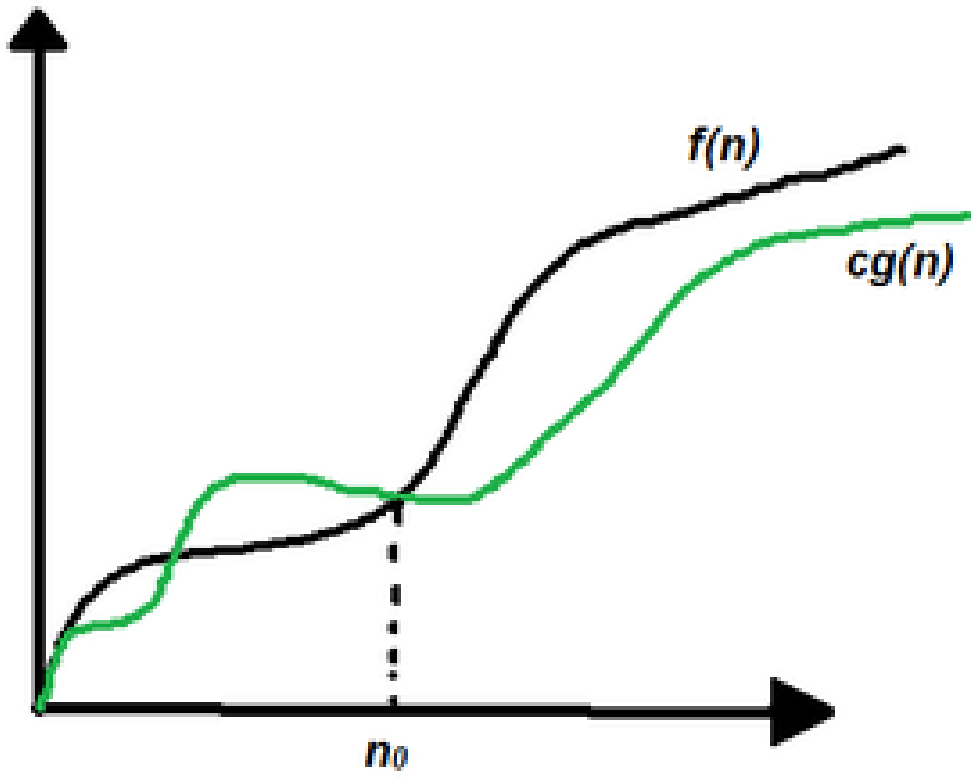
Omega notation

- **Omega notation** represents the lower bound of the running time of an algorithm.
- Thus, it provides the **best-case** complexity of an algorithm.
- The execution time serves as a **lower bound** on the algorithm's time complexity.
- It is defined as the condition that allows an algorithm to complete statement execution in the **shortest amount of time.**



Omega Notation

- **Mathematical Representation of Omega notation :**
- ***Ω(g(n))*** *= {* ***f(n):*** *there exist positive constants c and n0 such that* ***0 ≤ cg(n) ≤ f(n)*** *for all n ≥ n0 }*
  Note: *Ω (g) is a set*



The Complexity of an Algorithm

- The complexity can be seen in two important aspects:
- Time Complexity - How long does the algorithm takes to compute the output(s)
- Space Complexity - How much space does the algorithm use for computing the output(s).
- We can also think about the utilization of the above two facts.

Input size and basic operation examples

Problem	***Input size measure***	***Basic operation***
Search for key in list of n items	Number of items in list n	Key comparison
Multiply two matrices of floating-point numbers	Dimensions of matrices	Floating point multiplication
Compute a^n	n	Floating point multiplication
Graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

The Running Time of a Program

- Big-O tells us that *for large input sizes*, $f(n)$will not grow faster than some constant multiple of $g(n)$.
- $f(n) \leq c \cdot g(n)$for all $n \geq n_0$

- Example 1: Linear Search
- Running time: $f(n) = n + 3$
- Choose $g(n) = n$
- For $c = 2, n_0 = 3$
- $f(n)=n+3 \leq 2n$ for all $n \geq 3$
- Therefore, $f(n) \in O(n)$

Question 1

- Find the running time of $f(n) = 3n^2 + 5n + 20$

Asymptotic Analysis - Example

$$T(n)=13n^3+42n^2+2n\log n+4n$$

- As n grows larger, n^3 is MUCH larger than n^2, nlogn, and n, so it dominates (n^3)

- The running time grows roughly on the order of $T(n)=O(n^3)$

Ignoring constants in T(n) Analyzing T(n) as n "gets large

Worst-Case-Scenario

- Big-O also assumes the worst-case-scenario for an algorithm.
- Suppose an algorithm had to check each item in an input array and would return when it found what it was looking for.
- This could be represented as O(n), meaning that the number of operations would be equal to the number of inputs (items in the array).
- It is possible that the algorithm would return early, but in Big-O, we assume that it would have to check each item in the array (the worst-case-scenario).

Can we justify Big O notation?

- **Big O notation is a *huge* simplification; can we justify it?**
  - It only makes sense for *large* problem sizes
  - For sufficiently large problem sizes, the highest-order term swamps all the rest!
- **Consider $R = n^2 + 3n + 5$ as n varies:**

n = 0	n2 = 0	~~3n = 10~~ 0	5 = 5	R = 5
n = 10	n2 = 100	3n = 30	5 = 5	R = 135
n = 100	n2 = 10000	3n = 300	5 = 5	R = 10,305
n = 1000	n2 = 1000000	3n = 3000	5 = 5	R = 1,003,005
n = 10,000				R = 100,030,005
n = 100,000				R = 10,000,300,005

=10,000,300,005-10,000,000,000= 300,005

As a percentage 300,005/10,000,000,000*100=0.0030%

Lecture – 9 Complexity and Analysis of Algorithms - Part B

M.G.Noel A.S Fernando (PhD)

Professor

Dept. of Information Systems Engineering, UCSC

Definition

- When we compute the time complexity T(n) of an algorithm, we rarely get an exact result, just an estimate.
- In computer science we are typically only interested in how fast T(n) is growing as a function of the input size n.
- For example, if an algorithm increments each number in a list of length n, we might say: "This algorithm runs in $O(n)$ time and performs $O(1)$ work for each element".

- X=X+1
- X=X+1 repeat N times
- ……

Common time complexities

BETTER

- $O(1)$ constant time
- $O(\log n)$ log time
- $O(n)$ linear time
- $O(n \log n)$ log linear time
- $O(n^2)$ quadratic time
- $O(n^3)$ cubic time
- $O(2^n)$ exponential time

WORSE

Big-Oh Rules

If f(n) is a polynomial of degree d, then f(n) is $O(n^d)$, i.e.,

1. Drop lower-order terms
2. Drop constant factors

Use the smallest possible class of functions

- "2n is O(n)" instead of "2n is $O(n^2)$"

Use the simplest expression of the class

- "3n + 5 is O(n)" instead of "3n + 5 is O(3n)"

More Examples:

- $f(n) = 7n^4 + 3n^2 + 5n + 1000$ is $O(n^4)$
- $g(n) = 7n + 3n + 1000000$ is $O(n)$
- $h(n) = 7n(n + \log(n))$ is $O(n^2)$

Questions (2 and 3)

2. Using a suitable example, show the worst-case running time of an if-then-else statement.

3. Using a suitable example, show the worst-case running time of a simple loop program segment

Question 4

- Consider the following C program segment.

```
#include <stdio.h>
int main() {
    int n = 10; // example input size
    printf("Hello\n");
    for (int i = 0; i < n; i++)
        printf("%d\n", i);
    }
    return 0;
}
```

Calculate the worst-case time complexity of the above code.

Question 5 - Calculate the worst-case running time of the following program

Multiply for Nested Loops

Python code

```python
for i in range(n):
    for j in range(n):
        print(i, j)
```

C code

```c
#include <stdio.h>
int main() {
    int n = 5; // example input size
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            printf("%d %d\n", i, j);
        }
    }
    return 0;
}
```

Question 6 - Calculate the worst-case running time of the following program

Keep the Dominant Term

When there are multiple terms, keep the *highest order*:

Python Code

```
for i in range(n):
    for j in range(n):
        print(i, j)
for k in range(n):
    print(k)
```

C code

```
#include <stdio.h>
int main() {
    int n = 5; // example input size
for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d %d\n", i, j);
        }
    }
    // Separate loop: O(n)
    for (int k = 0; k < n; k++) {
        printf("%d\n", k);
    }
    return 0;
}
```

Question Number 7: Find the running time of the recursive factorial

fact(n) = n*fact(n-1) = n*(n-1)*fact(n-2) = … = n*(n-1)*(n-2)*…*fact(1) = n!

```
public int  fact(int n) {
        if  n<=1  return 1;
        else  return  n*fact(n-1);
}
```

Question 8 – Calculate the Running time of the following program segment.

```
int k=0;
for (int i=0; i<n; i++) {                          /* outer_loop */
        if  even(i)  { for (int j=0; j<n; j++)      /* inner_loop */
                                k++; };
        else  k *= 2;
};
```

Question 9

- Consider an algorithm whose running time is given by $T(n) = n - 1$. Using Big-O notation, determine the time complexity of the algorithm and explain why it is classified as linear

Question 10 :calculate the time complexity

- $T(n) = n^2/2 - n/2$.

Question 11

- Prove that the function $T(n) = 3n^3 + 20n^2 + 5$ belongs to $O(n^3)$.
- Find appropriate constants $c > 0$and $n_0 \geq 1$such that:
- $3n^3+20n^2+5 \leq c \cdot n^3$for all $n \geq n_0$.
- Show your calculation and justification.

Question 12

- Consider the functions $f(n) = n^3 + 20n^2 + 100n$ and $g(n) = n^3$.
- a) Using Big-O notation, determine whether $f(n) = O\big(g(n)\big)$.
- (b) If possible, provide constants $c > 0$and $n_0 \geq 1$such that the inequality
- f(n)≤c·g(n),for all n≥n0
- (c) Verify your choice of constants by substituting them into the inequality.
- **hint:** You can choose $c = 121$and $n_0 = 1$.

Question 13 :More Big-Oh questions

- Prove that $100n + 5 = O(n)$

Question 14

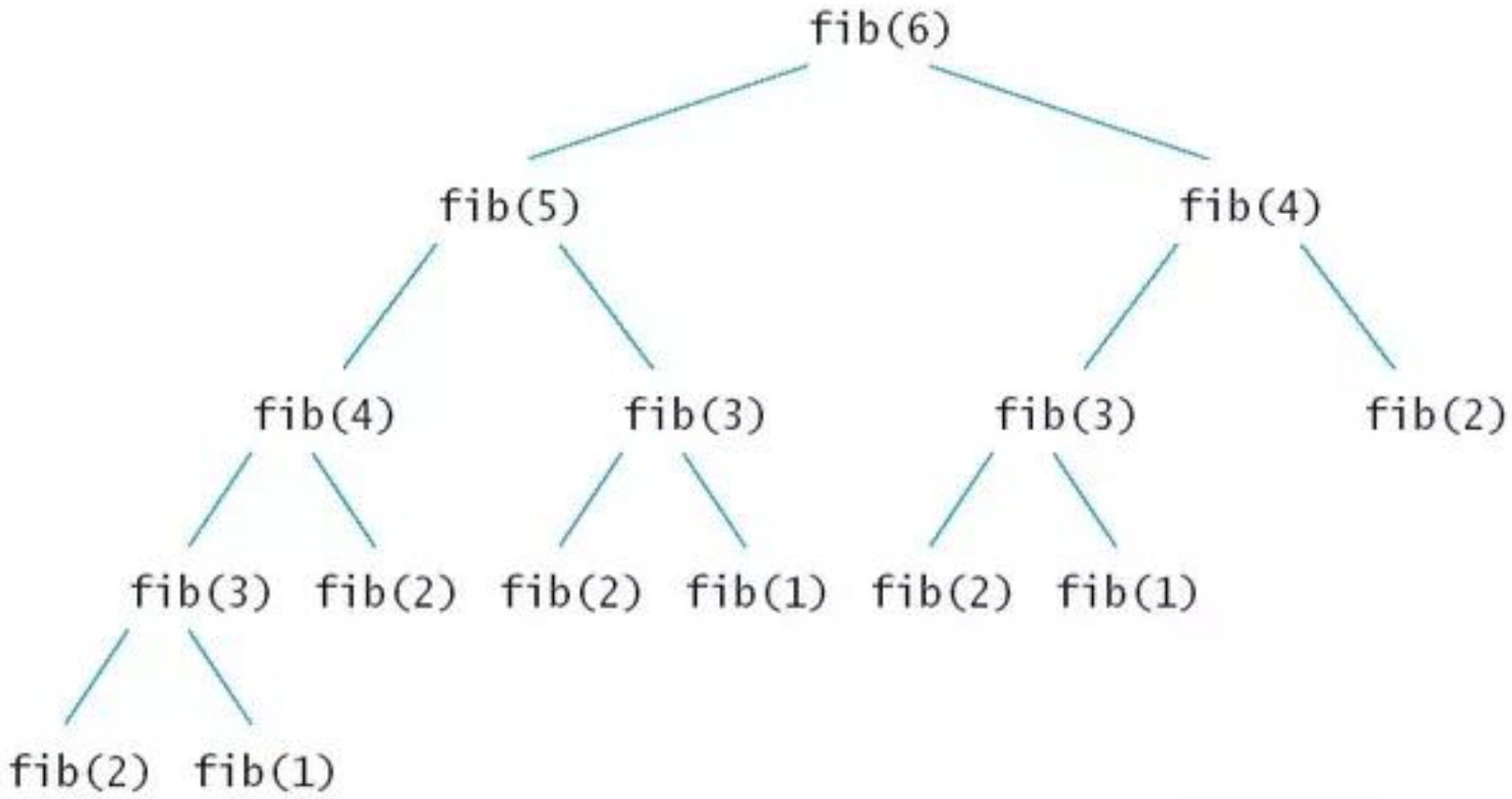
$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3$, $3n^2 > 3n^2 - 100n + 6$

Different big O values -Examples

- **Exponential Time — O(2^n)**
- An algorithm is said to have an exponential time complexity when the growth doubles with each addition to the input data set.
- A example of an exponential time algorithm is the recursive calculation of Fibonacci numbers:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Fibonacci numbers:



Fibonacci numbers:

$T(n<=1) = O(1)$

$T(n) = T(n-1) + T(n-2) + O(1)$

Base: $n = 1$ is obvious

O(1) – Constant

Assume $T(n-1) = O(2^{n-1})$, *therefore*

$T(n) = T(n-1) + T(n-2) + O(1)$ *which is equal to*

$T(n) = O(2^{n-1}) + O(2^{n-2}) + O(1) = O(2^{n})$

https://syedtousifahmed.medium.com/fibonacci-iterative-vs-recursive-5182d7783055

Fibonacci numbers:

- we can approximate $T(n-2) \sim T(n-1)$ as $T(n-2) \leq T(n-1)$
- $T(n) = T(n-1) + T(n-2) + c$

$= 2T(n-1) + c$ //from the approximation $T(n-1) \sim T(n-2)$

$=2*(2T(n-2) + c) + c$ // $T(n)=2T(n-1)+c$, similarly $T(n-1)=2T(n-2)+1$

$= 4T(n-2) + 3c$

$=8T(n-3) + 7c$

$=2^k * T(n - k) + (2^k - 1)*c$ /general formular

- Let's find the value of k for which: $n - k = 0$
  $k = n$

$T(n) = 2^n * T(0) + (2^n - 1)*c$

$=2^n*c+2^n*c$

i.e. $T(n) \sim 2^n$

Time Complexity Analysis of Searching Algorithms

- What are Searching Algorithms?
  - Searching algorithms are a class of algorithms in computer science and related fields that are designed to locate a specific item or group of items within a data structure (such as an array, list, tree, or database).

The following questions can be answered using searching algorithms:

- Does this element exist in the data?
- Where is this element located?
- How many times does it occur

Time Complexity Analysis of Searching Algorithms (cont..)

- Examples
- **Linear Search**
  - Checks each element one by one until it finds the target or reaches the end.
  - Works on unsorted data.
  - Time Complexity: *O(n)*.
- **Binary Search**
  - Works only on sorted data.
  - Repeatedly divides the search space in half to find the target.
  - Time Complexity: *O(log n)*.
- **Tree Search**
  - Used in tree data structures like Binary Search Trees (BSTs).
  - Efficiency depends on the tree's balance.

Linear Search Algorithm

- It is used for an unsorted array.
- It mainly does one by one comparison of the item to be search with array elements.
- It takes linear or O(n) Time.

Linear Search Algorithm

- Given an array, **arr** of n integers, and an integer element **x**, find whether element **x** is present in the array.
- Return the index of the first occurrence of **x** in the array, or -1 if it doesn't exist.
- ***Input***: *arr[] = [1, 2, 3, 4], x = 3*
  Output: *2*
  Explanation: *There is one test case with array as [1, 2, 3 4] and element to be searched as 3. Since 3 is present at index 2, the output is 2.*
- ***Input***: *arr[] = [10, 8, 30], x = 6*
  Output: *-1*
  Explanation: *The element to be searched is 6 and its not present, so we return -1.*

Linear Search Algorithm

1. Start from the first element.
2. Compare the current element with the target.
3. If it matches, return the index (position).
4. If not, move to the next element.
5. Repeat steps 2–4 until:
   - You find the element (success), or
   - You reach the end (failure).

List: [4, 2, 7, 1, 3]

Target: 1

Steps:

- Compare 4 → Not 1
- Compare 2 → Not 1
- Compare 7 → Not 1
- Compare 1 → Found at index 3

QUESTION 15

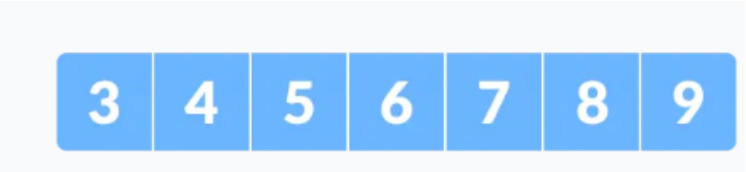
- Write pseudocode and a C program to search for an element in an array using linear search.
- Discuss the time complexity of the linear search algorithm

Binary search Algorithm

- Binary Search is a searching algorithm for finding an element's position in a sorted array.
- Binary Search Algorithm can be implemented in two ways:
- Iterative Method
- Recursive Method
- The recursive method follows the divide and conquer approach.

Binary search Algorithm

- The general steps for both methods are:
- **Step 1**: Consider the initial Array



  - Let x = 4 be the element to be searched.
- **Step 2**: Set two pointers low and high at the lowest and the highest positions respectively

Binary search Algorithm

- **Step 3**: Find the middle position mid of the array.
- i.e. mid = (low + high)/2 and arr[mid] = 6

Step 4: If x == arr[mid], then return mid. Else, compare the element to be searched with arr[mid].
Step 5: If x > arr[mid], compare x with the middle element of the elements on the right side of arr[mid]. This is done by setting low to low = mid + 1.
Step 6: Else, compare x with the middle element of the elements on the left side of arr[mid]. This is done by setting high to high = mid - 1

Binary search Algorithm

Step 7: Repeat steps 3 to 6 until low meets high

x = 4 is found

Question 16

a. Write (i) recursive (ii) iterative code for the binary search algorithm
b. Calculate the time complexity